

Audit Report: Assessment Attempts, Packages & Coupons

InterVu AI / SkillitriX Monorepo — System Interconnections & Refactoring Plan | Oct 7, 2026

1. Executive Summary

This read-only audit provides a comprehensive investigation into the storage, lifecycle, and runtime enforcement of maximum candidate assessment attempts across the codebase.

- **3 Divergent Configurations Found:** Assessment Configuration (RuleFlags.maxAttempts), Premium Package (PlanFeature.valueJson), and Assessment Coupon Code (ReferralCampaign / UserQuotaOverride).
- **Runtime Enforcement Priority:** Explicit entitlements (Package/Coupon attemptsPerExam) take top precedence. If absent or set to "all", the system falls back directly to RuleFlags.maxAttempts (default: 3).
- **The Pro Plan Attempt Ceiling Bug:** Paid Pro subscribers with "all" assessments access are currently silently capped by RuleFlags.maxAttempts (default 3) on individual exams.

2. Architecture & Relevant Modules

The attempt limit lifecycle spans 5 main subsystems:

- **Frontend Admin UI:** apps/web: rule-flags-tab.tsx (config editor), billing/plans/page.tsx (packages), billing/referrals/page.tsx (coupons).
- **Frontend Candidate UI:** apps/web: TestCard.tsx, TestDetailsPage.tsx, TestLaunchPage.tsx, CandidateHistorySection.tsx.
- **Backend Eligibility Gatekeeper:** apps/api: EligibilityService.ts, StartTestService.ts, EntitlementService.ts, UsageQuotaService.ts.
- **Referral & Coupon Engine:** apps/api: ReferralEngineService.ts, ReferralRewardService.ts, UserQuotaOverride repository.
- **Database Layer:** packages/database: RuleFlags, ExamPublishedVersion, PlanFeature, UserQuotaOverride, UsageQuota, TestInstance.

3. Inventory of Attempt-Related Fields

- **RuleFlags.maxAttempts:** PostgreSQL Int (@default(3), @map("max_attempts")) — Draft exam configuration limit.
- **ExamPublishedVersion.scoringRulesSnapshot:** JSON — Snapshot frozen at publishing time containing maxAttempts.
- **PlanFeature.valueJson:** JSON — Contains { attemptsPerExam, overallAttempts } or monthlyRoundsLimit.
- **UserQuotaOverride.overrideValue:** JSON — Granted on coupon redemption containing { attemptsPerExam, bonusRounds }.
- **UsageQuota.roundsUsed:** PostgreSQL Int (@default(0)) — Tracks monthly assessment quota consumed per candidate.
- **TestInstance (Records):** Table Rows — Each test attempt (CREATED, IN_PROGRESS, SUBMITTED) counts as 1 attempt.
- **Shared Contracts:** UpdateRuleFlags.maxAttempts (optional) & RuleFlagsResponseDto.maxAttempts (default: 10).

4. Assessment Configuration Attempt-Limit Flow

1. Admin edits RuleFlags in Admin UI (rule-flags-tab.tsx) -> Saved via PUT /exam-configs/:id/rules into RuleFlags.max_attempts.
2. On publishing, ConfigPublisherService copies ruleFlags.maxAttempts into ExamPublishedVersion.scoringRulesSnapshot.
3. When candidate starts test, EligibilityService checks testInstanceRepository.countAttempts against config.ruleFlags.maxAttempts (fallback).

5. Premium Package Attempt-Limit Flow

1. Admin configures PlanFeature under featureKey: "allowed_assessments" with { assessments: [...], attemptsPerExam: N }.
2. Candidate purchases plan -> Subscription created (status: ACTIVE) with monthlyRoundsLimit.
3. EligibilityService resolves allowedAssessments.attemptsPerExam as effectiveMaxAttempts.
4. On launch, EntitlementService.consumeRound() atomically increments UsageQuota.roundsUsed.

6. Coupon Code Attempt-Limit Flow

1. Admin creates ReferralCampaign with refereeRewardConfig: { featureKey: "allowed_assessments", overrideValue: { assessments: [...], attemptsPerExam: 1 } }.
2. Candidate redeems code -> ReferralEngineService creates UserQuotaOverride and ReferralRedemption.
3. EntitlementService checks completed TestInstance count. If count < attemptsPerExam, unlocks assessment and passes eligibility.

7. Candidate Attempt Lifecycle End-to-End

- Stage 1 (Fast-Path Active Test Check): If candidate has an in-progress attempt, returns existing testInstanceId (No quota consumed).
- Stage 2 (Entitlement & Limit Check): Resolves effectiveMaxAttempts. If previousAttempts >= effectiveMaxAttempts, throws ATTEMPT_LIMIT_REACHED.
- Stage 3 (Atomic Quota Consumption): Deducts 1 round from monthly quota. If exhausted, throws QUOTA_EXHAUSTED.
- Stage 4 (Creation): Creates TestInstance record in database.
- Stage 5 (Submission & Evaluation): Updates status to SUBMITTED and evaluates results.

8. Runtime Source-of-Truth Matrix

- **Package with specific attemptsPerExam:** Enforces Package attemptsPerExam (e.g. 5 attempts). Config limit ignored.
- **Package with all-access ("all"):** Falls back to RuleFlags.maxAttempts (e.g. 3 attempts) — Hazard Identified.
- **Coupon Only (e.g. MNC2026):** Enforces Coupon attemptsPerExam (e.g. 1 attempt). Config limit ignored.
- **Both Package and Coupon Active:** Merges using Math.max(packageAttempts, couponAttempts) and adds bonus rounds.
- **No Package & No Coupon:** Blocked immediately with QUOTA_EXHAUSTED.
- **Resume Active Attempt:** Active session returned seamlessly without consuming quota.

9. Confirmed Bugs & Critical Risks

CRITICAL: All-Access Pro Plan Attempt Ceiling

Paid Pro/Teams plans set `allowedAssessments: ["all"]` without specifying `attemptsPerExam`. Consequently, `EligibilityService` falls back to `RuleFlags.maxAttempts` (default 3), blocking paying candidates from their 4th attempt on an exam even though they have 20 monthly rounds remaining.

RISK: Abandoned Incomplete Attempts Counted

`testInstanceRepository.countAttempts` counts all `TestInstance` rows regardless of status. An abandoned attempt after 2 seconds permanently consumes an attempt slot.

10. Proposed File-by-File Change Plan

To cleanly decouple attempt limits from Assessment Configuration:

- **`apps/web/.../rule-flags-tab.tsx`**: Remove `maxAttempts` input field and Zod validation.
- **`apps/web/src/store/config-rules.store.ts`**: Remove `maxAttempts` from Zustand draft store.
- **`packages/shared/.../rule-flags contracts`**: Remove or deprecate `maxAttempts` in update and response DTOs.
- **`apps/api/.../config-publisher.service.ts`**: Remove `maxAttempts` from `scoringRulesSnapshot`.
- **`apps/api/.../eligibility.service.ts`**: Derive `effectiveMaxAttempts` strictly from Entitlements (Package/Coupon). Default to unlimited (null) if plan is all-access.
- **`apps/api/.../candidate-dashboard.service.ts`**: Calculate candidate dashboard attempt limits purely from user entitlements.
- **`apps/web/.../TestDetailsPage.tsx` & `TestCard.tsx`**: Read `maxAttempts` from `availableTest` / entitlement object.
- **`packages/database/prisma/schema.prisma`**: Phase 2: Deprecate or drop `max_attempts` column on `RuleFlags`.

11. Testing & Verification Strategy

1. Verify Pro plan subscribers can take unlimited attempts per exam up to their monthly quota budget.
2. Verify package-specific attempt limits (e.g. 5 attempts) are strictly enforced.
3. Verify coupon code single-attempt access (e.g. MNC2026) operates without error.
4. Verify concurrent test launch race conditions are prevented by atomic `consumeRound`.
5. Verify resume operations do not increment attempt counts.

12. Conclusion & Product Owner Decisions Required

Decisions Required from Product Owner

1. Unlimited Attempts Policy: Confirm Pro/Teams all-access subscribers should have unlimited attempts per exam (capped only by monthly rounds quota).
2. Incomplete Attempts: Confirm whether abandoned/unsubmitted tests should count against the attempt limit or be refunded upon expiry.
3. Free Assessments: Confirm default access policy for free mock tests without active coupons.

